THE SUBWORD COMPLEXITY POLYNOMIAL $S(n,x)$ HAS LEADING COEFFICIENT $n(n+1)/2$: A PROOF OF GUO'S CONJECTURE (OEIS A341761)

MATH0

Abstract. For a finite word w let $c(w)$, the subword complexity of w, be the number of distinct nonempty factors (contiguous subwords) of w, and let $S(n,x)$ be the sum of $c(w)$ over the x^n words w of length n over an alphabet with x letters. In OEIS A341761, Shiyao Guo conjectured in 2021 that the coefficient of x^n in the polynomial $S(n,x)$ is $n(n+1)/2$. We prove this by grouping words according to which positions carry equal letters: $S(n,x) = \sum_p c(p)\, x(x-1)\cdots(x-k(p)+1)$ over the equality patterns p, where $k(p)$ is the number of distinct letters, and only the pattern with n distinct letters contributes to x^n. The argument is elementary; we found no earlier proof in the literature and report the result as new, settling the conjecture as stated. The proof, including the polynomiality of $S(n,\cdot)$, is verified in Lean 4 against the frozen, fidelity-reviewed statement that math0 formalized from the OEIS entry; the kernel checks that statement's definition of S against rows 0–5 of the OEIS data.

1. Introduction

A *factor* of a word $w = w_1 w_2 \cdots w_n$ is a contiguous subword $w_i w_{i+1} \cdots w_j$ with $1 \le i \le j \le n$. The *subword complexity* $c(w)$ is the number of distinct nonempty factors of w; for example $c(aba) = 5$, the factors being a, b, ab, ba, aba. For $n, x \ge 0$ let $S(n,x) = \sum_{w \in A_x^n} c(w)$, where A_x is an alphabet with x letters. For $x \ge 1$, $S(n,x)/x^n$ is the expected subword complexity of a uniformly random word of length n over A_x.

The OEIS entry A341761 [10], submitted by Shiyao Guo on February 19, 2021, is the triangle whose row n lists the coefficients of $S(n,x)$ as a polynomial in x, constant term first: $0;\ 0,1;\ 0,-1,3;\ 0,0,-3,6;\ 0,-1,1,-6,10;\ \ldots$ The entry states, without proof, that $S(n,x)$ has degree n and constant coefficient 0, and it contains the conjecture that the coefficient of x^n is $n(n+1)/2$. The companion entry A340885 [9] lists $S(n,2)$. In a blog post [4], Guo computes these polynomials (written $F_n(m)$ there) for $n \le 60$ by grouping candidate factors according to their autocorrelation, notes that F_n is a polynomial of degree at most n, and lists $[x^n]F_n(x) = n(n+1)/2$ with the remark "This is a conjecture." The entry links a table of rows 0–60.

We prove the conjecture (Theorem 1), via an explicit expansion of $S(n,x)$ in falling factorials of x (Proposition 7).

Prior work and status. The conjecture is still labelled "Conjecture" in the current revision of A341761 (#33, March 20, 2024). Guo's blog post [4], in its English and Chinese versions (updated March 26, 2026), still calls it a conjecture; it justifies $\deg F_n \le n$ in one line but gives no argument for the leading coefficient. The average subword complexity of random words has been studied by Janson, Lonardi and Szpankowski [6] (the average number of distinct substrings of a random text of length n over a k-letter alphabet), Ivanko [5] (an explicit approximation over a finite alphabet), Gheorghiciuc and Ward [3] (the expected number of distinct factors of a fixed length, via correlation polynomials) and Ahmadi and Ward [1] (its asymptotics for binary

Date: October 24, 2026.

memoryless sources). These works concern the behaviour as the word length grows, for a fixed alphabet or source; as far as the math0 referees could determine, none addresses the coefficients of $S(n,x)$ as a polynomial in the alphabet size. The referees' searches (arXiv, Mathematics Stack Exchange, Chinese-language sources, general web search) found neither a proof nor a counterexample; the SeqFan mailing-list archive could not be searched directly.

We therefore report the result as new. We stress that it is elementary. Once one knows that $S(n,\cdot)$ is a polynomial of degree at most n (which Guo notes), its coefficient of x^n equals $\lim_{x\to\infty} S(n,x)/x^n$, and a uniformly random word of length n has pairwise distinct letters with probability $x(x-1)\cdots(x-n+1)/x^n \to 1$; such a word has $n(n+1)/2$ distinct factors (Lemma 8). Specialists may regard the statement as evident; we found no recorded proof. The Lean proof establishes Theorem 1, including the existence of the polynomial, and on the way the formula of Proposition 7. Remark 9 is proved on paper only.

2. Statement

The COMMENTS section of A341761 [10] reads, in part (verbatim):

> S(n,x) is the sum of subword complexities (number of nonempty distinct subwords) of all words of length n and an alphabet with size x.
> [...]
> Conjecture: the coefficient of x^n in S(n,x) is n*(n+1)/2.

These lines carry no separate signature; they belong to the original submission, whose author line reads _Shiyao Guo_, Feb 19 2021.

Theorem 1. *For every $n \geq 0$ there is a polynomial $P_n \in \mathbb{Q}[X]$ with $P_n(x) = S(n,x)$ for every $x \in \mathbb{N}$, and the coefficient of X^n in P_n is $n(n+1)/2$.*

Such a polynomial is unique, since two polynomials over $\mathbb{Q}$ that agree on $\mathbb{N}$ are equal (`poly_unique` below); so Theorem 1 is a statement about *the* polynomial $S(n,x)$.

Lean formalization. The statement was formalized by math0 in `lean/Math0/Statements/OEIS/A341761.lean`, reviewed for fidelity, and frozen (SHA-256 `827988f4...faf23`) before any proof attempt. Its definitions and the statement read (verbatim; header, import, docstrings, proofs, one helper lemma and the data-check declarations omitted):

```
namespace Math0.Statements.A341761
open Polynomial
def factors {α : Type*} [DecidableEq α] (w : List α) : Finset (List α) :=
  (w.tails.flatMap fun t => t.inits.tail).toFinset
theorem mem_factors {α : Type*} [DecidableEq α] (w u : List α) :
    u ∈ factors w ↔ u ≠ [] ∧ u <:+: w
def complexity {α : Type*} [DecidableEq α] (w : List α) : ℕ := (factors w).card
def S (n x : ℕ) : ℕ := ∑ w : Fin n → Fin x, complexity (List.ofFn w)
theorem poly_unique (P Q : ℚ[X]) (h : ∀ x : ℕ, P.eval (x : ℚ) = Q.eval (x : ℚ)) : P = Q
def statement : Prop :=
  ∀ n : ℕ, ∃ P : ℚ[X], (∀ x : ℕ, P.eval (x : ℚ) = (S n x : ℚ)) ∧
    P.coeff n = (n : ℚ) * (n + 1) / 2
end Math0.Statements.A341761
```

The encoding is as follows.

(1) *Factors.* `factors w` is the set of nonempty prefixes of the suffixes of w, and `mem_factors` proves that it consists exactly of the nonempty lists u that are contiguous infixes of w (`<:+:` is Mathlib's infix relation). "Subword" in the entry means a contiguous factor: the entry's example lists the complexities $3,5,5,5,5,5,5,3$ of the binary words of length 3,

and $c(aba) = 5$ excludes the scattered subsequence aa. `complexity w` is the number of these factors, i.e. $c(w)$.

(2) $S(n, x)$. The words of length n over an alphabet of size x are the maps `Fin` $n \to$ `Fin` x, turned into lists by `List.ofFn`; thus $S(0, x) = 0$ (the empty word has complexity 0) and $S(n, 0) = 0$ for $n \geq 1$, consistent with the entry.

(3) *The polynomial.* "The coefficient of x^n in $S(n, x)$" is read as the coefficient of X^n in a polynomial $P \in \mathbb{Q}[X]$ that agrees with $S(n, \cdot)$ on $\mathbb{N}$. By `poly_unique`, two polynomials over $\mathbb{Q}$ agreeing on $\mathbb{N}$ are equal, so such a P is unique, and the existential quantifier adds only the polynomiality that the entry asserts. Coefficients are rational, so no integrality is assumed; $n(n+1)/2$ is computed in $\mathbb{Q}$.

Check against the OEIS data. The statement file also defines the triangle and checks it against the entry, through the following declarations (verbatim; proofs, auxiliary lemmas, and the definitions of `interp`, `a`, and of the list `words x n` of all words of length n over `Fin x` omitted):

```
def Sw (n x : ℕ) : ℕ := ((words x n).map complexity).sum
theorem S_eq_Sw : S = Sw
def row (n : ℕ) : List ℚ := interp ((List.range (n + 1)).map fun x => (S n x : ℚ))
theorem rows_data : (List.range 6).map row =
    [[0], [0, 1], [0, -1, 3], [0, 0, -3, 6], [0, -1, 1, -6, 10], [0, 2, -6, 4, -10, 15]]
theorem a_data : (List.range 21).map (fun i => a (i + 0)) =
    [0, 0, 1, 0, -1, 3, 0, 0, -3, 6, 0, -1, 1, -6, 10, 0, 2, -6, 4, -10, 15]
```

Here `interp` computes, by Newton's forward-difference formula $P(x) = \sum_{k \leq n} \Delta^k v_0 \binom{x}{k}$ over $\mathbb{Q}$, the coefficients of the polynomial of degree at most n through the values $v_x = S(n, x)$, $x = 0, \ldots, n$; this recovers the coefficients of $S(n, x)$ given the entry's claim that its degree is n, and `a` reads the rows in order (offset 0). Through `S_eq_Sw`, `rows_data` is proved by `decide +kernel`: the kernel enumerates all x^n words for $x \leq n \leq 5$, computes their complexities and interpolates (about 90 seconds of CPU time); `a_data` then confirms the first 21 terms of the OEIS data. The theorem we prove refers only to S; the data check anchors the definition of S to the entry independently of the conjecture. Outside Lean, the formula of Proposition 7, evaluated over all equality patterns with $n \leq 10$, reproduces all 66 terms of the OEIS data section (rows 0–10).

3. Proof

The proof follows the Lean file `lean/Math0/Frontier/OEIS/A341761.lean`; Lean names are given in brackets. *Notation.* For $n \in \mathbb{N}$ let $[n] = \{0, 1, \ldots, n-1\}$. A word of length n over a set A is a map $w \colon [n] \to A$, identified with the list $w(0)w(1)\cdots w(n-1)$. We write $\mathcal{F}(u)$ for the set of distinct nonempty factors of a word u, so $c(u) = |\mathcal{F}(u)|$. For a map $f \colon A \to B$, $f(u)$ denotes the letterwise image of u. For $x, k \in \mathbb{N}$ let $x^{\underline{k}} = x(x-1)\cdots(x-k+1)$ (`Nat.descFactorial`; it is 0 if $k > x$), and let $X^{\underline{k}} = X(X-1)\cdots(X-k+1) \in \mathbb{Q}[X]$ (`descPochhammer ℚ k`).

3.1. Equality patterns.

Definition 2 (`pat`)**.** For $w \colon [n] \to A$ and $i \in [n]$ let $\pi_w(i) = \min\{j \in [n] : w(j) = w(i)\}$, the first position carrying the letter $w(i)$. A map $p \colon [n] \to [n]$ is a *pattern* if $\pi_p = p$. For a pattern p let $\mathrm{Fix}(p) = \{i : p(i) = i\}$ and $k(p) = |\mathrm{Fix}(p)|$ (`nfix`).

Lemma 3 (`pat_spec`, `pat_eq_iff`, `pat_congr`, `pat_pat`, `pat_idem_apply`, `pat_id`)**.** *Let* $w \colon [n] \to A$.

(1) $w(\pi_w(i)) = w(i)$ *for all* i.

(2) $\pi_w(i) = \pi_w(j)$ *if and only if* $w(i) = w(j)$.
(3) *If* $v\colon [n] \to B$ *satisfies* $w(i) = w(j) \Leftrightarrow v(i) = v(j)$ *for all* i, j, *then* $\pi_v = \pi_w$.
(4) π_w *is a pattern.*
(5) *If* p *is a pattern, then* $p(p(i)) = p(i)$ *for all* i. *The identity map of* $[n]$ *is a pattern.*

Proof. (1) The minimum is taken over a set that contains i, and it belongs to that set. (2) If $\pi_w(i) = \pi_w(j)$, then $w(i) = w(\pi_w(i)) = w(\pi_w(j)) = w(j)$ by (1). If $w(i) = w(j)$, the two sets whose minima define $\pi_w(i)$ and $\pi_w(j)$ coincide. (3) For each i the sets $\{j : w(j) = w(i)\}$ and $\{j : v(j) = v(i)\}$ coincide. (4) By (2), π_w and w satisfy the hypothesis of (3), so $\pi_{\pi_w} = \pi_w$. (5) Applying (1) to $w = p$ gives $p(\pi_p(i)) = p(i)$, and $\pi_p = p$. For the identity, (1) gives $\pi_{\mathrm{id}}(i) = \mathrm{id}(\pi_{\mathrm{id}}(i)) = \mathrm{id}(i) = i$. □

3.2. **Complexity depends only on the pattern.**

Lemma 4 (`complexity_map`, `map_injOn_lists`)**.** *Let* u *be a word over* A *and let* $f\colon A \to B$ *be injective on the set of letters of* u. *Then* $c(f(u)) = c(u)$.

Proof. We show that $v \mapsto f(v)$ is a bijection from $\mathcal{F}(u)$ onto $\mathcal{F}(f(u))$. The image $f(v)$ of a factor v of u is a factor of $f(u)$, nonempty if v is. Conversely, every factor of $f(u)$ has the form $f(v)$ for a factor v of u (Mathlib's `List.infix_map_iff`), and v is nonempty if $f(v)$ is. So $\mathcal{F}(f(u)) = \{f(v) : v \in \mathcal{F}(u)\}$. The letters of every $v \in \mathcal{F}(u)$ are letters of u. If $f(v) = f(v')$ for $v, v' \in \mathcal{F}(u)$, then v and v' have the same length and, f being injective on the letters of u, the same letter at each position; so $v = v'$ (by induction on the lists in Lean). □

Lemma 5 (`complexity_pat`)**.** *For every word* $w\colon [n] \to A$, $c(w) = c(\pi_w)$, *where* π_w *is regarded as a word over* $[n]$.

Proof. By Lemma 3(1), $w = w \circ \pi_w$; that is, the word w is the letterwise image of the word π_w under the map $w\colon [n] \to A$. The letters of π_w are the positions $\pi_w(i)$, and w is injective on them: if $w(\pi_w(i)) = w(\pi_w(j))$, then $w(i) = w(j)$ by Lemma 3(1), hence $\pi_w(i) = \pi_w(j)$ by Lemma 3(2). Now apply Lemma 4. □

3.3. **Counting the words with a given pattern.**

Lemma 6 (`fiberEquiv`, `card_fiber`)**.** *Let* $x \in \mathbb{N}$ *and let* $p\colon [n] \to [n]$.

(1) *If* p *is a pattern, the number of words* $w\colon [n] \to [x]$ *with* $\pi_w = p$ *is* $x^{\underline{k(p)}}$.
(2) *If* p *is not a pattern, there is no word* w *with* $\pi_w = p$.

Proof. (1) Let $W_p = \{w\colon [n] \to [x] : \pi_w = p\}$ and let E be the set of injective maps $\mathrm{Fix}(p) \to [x]$. For $w \in W_p$ let $\rho(w)$ be the restriction of w to $\mathrm{Fix}(p)$. It is injective: if $i, j \in \mathrm{Fix}(p)$ and $w(i) = w(j)$, then $p(i) = \pi_w(i) = \pi_w(j) = p(j)$ by Lemma 3(2), so $i = p(i) = p(j) = j$. For $e \in E$ let $\sigma(e) = e \circ p$, which is defined because $p(i) \in \mathrm{Fix}(p)$ by Lemma 3(5). Since e is injective, $\sigma(e)(i) = \sigma(e)(j)$ if and only if $p(i) = p(j)$, so $\pi_{\sigma(e)} = \pi_p = p$ by Lemma 3(3); thus $\sigma(e) \in W_p$. The maps are mutually inverse: $\sigma(\rho(w))(i) = w(p(i)) = w(\pi_w(i)) = w(i)$ by Lemma 3(1), and $\rho(\sigma(e))(i) = e(p(i)) = e(i)$ for $i \in \mathrm{Fix}(p)$. Hence $|W_p| = |E|$, and the number of injections from a k-element set into an x-element set is $x^{\underline{k}}$ (Mathlib's `Fintype.card_embedding_eq`). (2) If $\pi_w = p$, then $\pi_p = \pi_{\pi_w} = \pi_w = p$ by Lemma 3(4). □

Proposition 7 (`S_eq`)**.** *For all* $n, x \in \mathbb{N}$,

$$S(n, x) = \sum_{p \ \mathit{pattern}} c(p)\, x^{\underline{k(p)}},$$

the sum running over the patterns $p\colon [n] \to [n]$.

Proof. Partition the words $w\colon [n] \to [x]$ according to π_w (`Finset.sum_fiberwise`). By Lemma 5, every word w with $\pi_w = p$ has $c(w) = c(p)$. So the words with $\pi_w = p$ contribute $c(p)$ times their number, which is $x^{\underline{k(p)}}$ if p is a pattern and 0 otherwise, by Lemma 6. □

For example, the five patterns for $n = 3$ correspond to aaa, aab, aba, abb, abc, with $(k, c) = (1, 3), (2, 5), (2, 5), (2, 5), (3, 6)$, so $S(3, x) = 3x + 15\,x(x-1) + 6\,x(x-1)(x-2) = 6x^3 - 3x^2$, which is row 3 of the triangle.

3.4. **Words with distinct letters.**

Lemma 8 (`complexity_nodup`, `card_inits_tail`)**.** *If u is a word of length m with pairwise distinct letters, then $c(u) = m(m+1)/2$.*

Proof. We prove $2c(u) = m(m+1)$ by induction on u. The empty word has no nonempty factors. Let $u = au'$ where a does not occur in u', the letters of u' are pairwise distinct, and u' has length m'. Since the suffixes of u are u itself and the suffixes of u', the definition of `factors` gives $\mathcal{F}(u) = \Pi \cup \mathcal{F}(u')$, where Π is the set of nonempty prefixes of u. The union is disjoint: every element of Π begins with a, while every factor of u' consists of letters of u'. The $m'+1$ nonempty prefixes of u have different lengths, so $|\Pi| = m' + 1$. Hence $2c(u) = 2(m' + 1) + m'(m' + 1) = (m' + 1)(m' + 2)$. □

3.5. **Proof of Theorem 1.** Fix n and set

$$P_n = \sum_{p \text{ pattern}} c(p)\, X^{\underline{k(p)}} \in \mathbb{Q}[X].$$

(In Lean the sum runs over all $p\colon$ `Fin` $n \to$ `Fin` n, with summand 0 when p is not a pattern.) For $x \in \mathbb{N}$ we have $X^{\underline{k}}(x) = x^{\underline{k}}$ (`descPochhammer_eval_eq_descFactorial`), so $P_n(x) = S(n, x)$ by Proposition 7.

For the coefficient of X^n, recall that $X^{\underline{k}}$ is monic of degree k (`monic_descPochhammer`, `descPochhammer_natDegree`). If p is a pattern other than the identity, then $p(i) \neq i$ for some i, so $\mathrm{Fix}(p)$ is a proper subset of $[n]$, $k(p) < n$, and $X^{\underline{k(p)}}$ has coefficient 0 at X^n. The identity is a pattern (Lemma 3(5)) with $k = n$, and $X^{\underline{n}}$ has coefficient 1 at X^n. Hence (`Finset.sum_eq_single`)

$$[X^n]\, P_n = c(0\,1 \cdots (n-1)).$$

The word $0\,1 \cdots (n-1)$ has pairwise distinct letters, so by Lemma 8 this equals $n(n+1)/2$. □

Remark 9 (not formalized)**.** Each $X^{\underline{k}}$ has integer coefficients, and for $n \geq 1$ every pattern fixes 0 (as $\pi_p(0) = 0$), so $k(p) \geq 1$ and X divides $X^{\underline{k(p)}}$. Hence, for $n \geq 1$, $P_n \in \mathbb{Z}[X]$ has degree exactly n and constant coefficient 0, as the entry states, and its coefficients in the basis $X^{\underline{0}}, \ldots, X^{\underline{n}}$ are the nonnegative integers $\sum_{k(p)=k} c(p)$.

4. Machine verification

Files and declarations. The proof is in `lean/Math0/Frontier/OEIS/A341761.lean` in the math0 repository, `https://github.com/Foundation-0/math0`. It imports the frozen statement file `lean/Math0/Statements/OEIS/A341761.lean` and the audit commands in `lean/Math0/Audit.lean`. The main declaration is `Math0.Frontier.A341761.result`, of type `Math0.Statements.A341761.statement`; the private auxiliary declarations are named as in Section 3, plus two small list lemmas (`toFinset_map'`, `card_inits`). The pattern map and the start of the main proof, which supplies P_n, read (verbatim excerpts; the last line is our summary of the omitted rest):

```
private def pat {n : ℕ} {α : Type*} [DecidableEq α] (w : Fin n → α) (i : Fin n) : Fin n :=
  (univ.filter fun j => w j = w i).min' ⟨i, by simp⟩
private def nfix {n : ℕ} (p : Fin n → Fin n) : ℕ := (univ.filter fun i => p i = i).card
theorem result : statement := by
  intro n
  refine ⟨∑ p : Fin n → Fin n, if pat p = p then
      C (complexity (List.ofFn p) : ℚ) * descPochhammer ℚ (nfix p) else 0, ?_, ?_⟩
  -- (evaluation at x : ℕ via S_eq; coefficient of X^n via Finset.sum_eq_single id)
```

Neither file uses `sorry`, `native_decide` or custom axioms.

Audit. The file ends with the command `#math0_settles` applied to `result` and `statement`. This command, defined in `lean/Math0/Audit.lean` for statements that math0 formalizes itself, fails unless (1) `statement` is a `Prop`-valued definition without universe parameters; (2) `statement` and `result` depend on no axioms besides `propext`, `Classical.choice` and `Quot.sound` (so no `sorry`, `native_decide` or custom axiom); and (3) the type of `result` is syntactically `statement` or `¬ statement`, up to metadata annotations, with no unfolding. Here it reports

```
math0 audit: 'Math0.Statements.A341761.statement' depends on axioms [Classical.choice, Quot.sound, propext]
math0 audit: 'Math0.Frontier.A341761.result' depends on axioms [Classical.choice, Quot.sound, propext]
math0 statement check: 'Math0.Frontier.A341761.result' proves 'Math0.Statements.A341761.statement'
```

Toolchain and reproduction. We use Lean 4 [8] `v4.33.1` and Mathlib [7] `v4.33.1`, pinned in `lean/`; formal-conjectures [2] is pinned at commit `c77fdd4` but not used here, as the statement is a math0 formalization of an OEIS conjecture. To reproduce, run `scripts/verify.sh` from the repository root: it fetches the Mathlib build cache, rebuilds all math0 results (re-running every audit command) and checks that the pinned dependencies are unmodified. The proof file alone is checked by `lake env lean Math0/Frontier/OEIS/A341761.lean` in `lean/` (about 15 seconds of CPU time once the statement file is built).

References

[1] L. Ahmadi and M. D. Ward, *Asymptotic analysis of the kth subword complexity*, Entropy **22** (2020), no. 2, 207, https://doi.org/10.3390/e22020207.

[2] M. Firsching, P. Lezeau, S. Mercuri, M. Z. Horváth, Y. Dillies, C. Sönne, E. Wieser, F. Zhang, T. Hubert, B. Agüera y Arcas, and P. Kohli, *Formal Conjectures: An Open and Evolving Benchmark for Verified Discovery in Mathematics*, arXiv:2605.13171 [cs.AI], 2026. Repository: https://github.com/google-deepmind/formal-conjectures.

[3] I. Gheorghiciuc and M. D. Ward, *On correlation polynomials and subword complexity*, in: 2007 Conference on Analysis of Algorithms (AofA 07), Discrete Math. Theor. Comput. Sci. Proc. **AH** (2007), 1–18, https://doi.org/10.46298/dmtcs.3553.

[4] S. Guo (Mivik), *On the expected subword complexity of random words*, blog post, January 30, 2021, updated March 26, 2026, https://mivik.moe/2021/research/expected-subword-complexity-en/ (Chinese version: .../expected-subword-complexity/); linked from A341761 via mivik.gitee.io.

[5] E. E. Ivanko, *Exact approximation of average subword complexity of finite random words over finite alphabet*, Trudy Inst. Mat. i Mekh. UrO RAN **14** (2008), no. 4, 185–189.

[6] S. Janson, S. Lonardi, and W. Szpankowski, *On average sequence complexity*, Theoret. Comput. Sci. **326** (2004), 213–227.

[7] The mathlib Community, *The Lean mathematical library*, in: Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020), ACM, 2020, 367–381.

[8] L. de Moura and S. Ullrich, *The Lean 4 theorem prover and programming language*, in: Automated Deduction – CADE 28, Lecture Notes in Comput. Sci. **12699**, Springer, 2021, 625–635.

[9] The OEIS Foundation Inc., *Sum of subword complexity (number of nonempty distinct subwords) of all binary strings of length n*, entry A340885 (S. Guo, 2021) in The On-Line Encyclopedia of Integer Sequences, https://oeis.org/A340885.

[10] The OEIS Foundation Inc., *Triangle read by rows in which row n is the coefficients of the subword complexity polynomial* $S(n, x)$, entry A341761 (S. Guo, 2021) in The On-Line Encyclopedia of Integer Sequences, `https://oeis.org/A341761`; revision #33, March 20, 2024.

Produced by Claude (an AI model developed by Anthropic) in the math0 project